RFLens: Agentic RF Data Platform (Project Blueprint)

Oct 24, 2026 · @Rohit

1. Project at a glance

RFLens lets a wireless engineer ask plain-English questions across real RF compliance data and test reports, and get answers that cite the SQL rows and document passages they came from.

Item	Details
What it is	An agentic data platform: LangGraph agents calling three MCP servers over Postgres with pgvector
Domain	Wireless and RF engineering data: FCC equipment authorizations, 3GPP specs, spectrum licences, signal and manufacturing-yield datasets
Unique angle	An extraction agent turns FCC test-report PDFs (conducted power, spurious emissions, margin against limit) into structured Postgres tables, so cross-device RF questions become SQL
Why it fits the JD	It covers every listed responsibility: multi-LLM integration including on-prem, tool use, RAG, multi-agent coordination with persistent state, and production deployment
How it differs from your current projects	AgentRAG and your MCP server are the base skills; this adds Postgres, text-to-SQL, report extraction, persistent state, provider fallback and load testing
Honest framing	A public-data analogue of the Smart Data Ecosystem problem. It uses no Apple data, and the README should say so

2. JD to build map

Each JD line gets one component and one artifact you can show.

JD requirement	What you build	Proof to show
Integrate LLMs (Claude, Mistral, Gemini) via cloud APIs and secure on-prem	A provider gateway switched by config, with timeouts, retries, fallback, and an on-prem path through vLLM or Ollama	A table of the same golden set run on three cloud models and one local model
Tool use and integration; query high-dimensional databases; custom engineering tools	Three MCP servers: sql-mcp, docs-mcp, analytics-mcp, with guardrails	Tool-call accuracy and an unsafe-SQL rejection test
Advanced RAG and fine-tuning	Hybrid retrieval in Postgres with citations; an optional fine-tuned embedder	recall@5 and citation precision, with a before and after for the fine-tune
Multi-agent design with orchestration, coordination, persistent state	LangGraph agents with a Postgres checkpointer; pause for approval, resume after restart	A demo that kills the service mid-investigation and resumes it
Scalable deployment from prototype to production	Docker compose, Kubernetes manifests, stateless services, load tests	p50 and p95 latency at stated concurrency, with 1 vs N replicas
Postgres, Aurora or BigQuery experience	Postgres 16 with pgvector, partitioned tables, indexes, EXPLAIN-based query limits	Schema, migrations and a query-latency table
MCPs, agents, CLIs, tool calling	A CLI (rflens ask) using the same MCP servers as the UI	CLI recording in the README

3. Real public data

Five public sources cover the structured, document and signal sides. I checked each access path; the catch column says what I could not confirm.

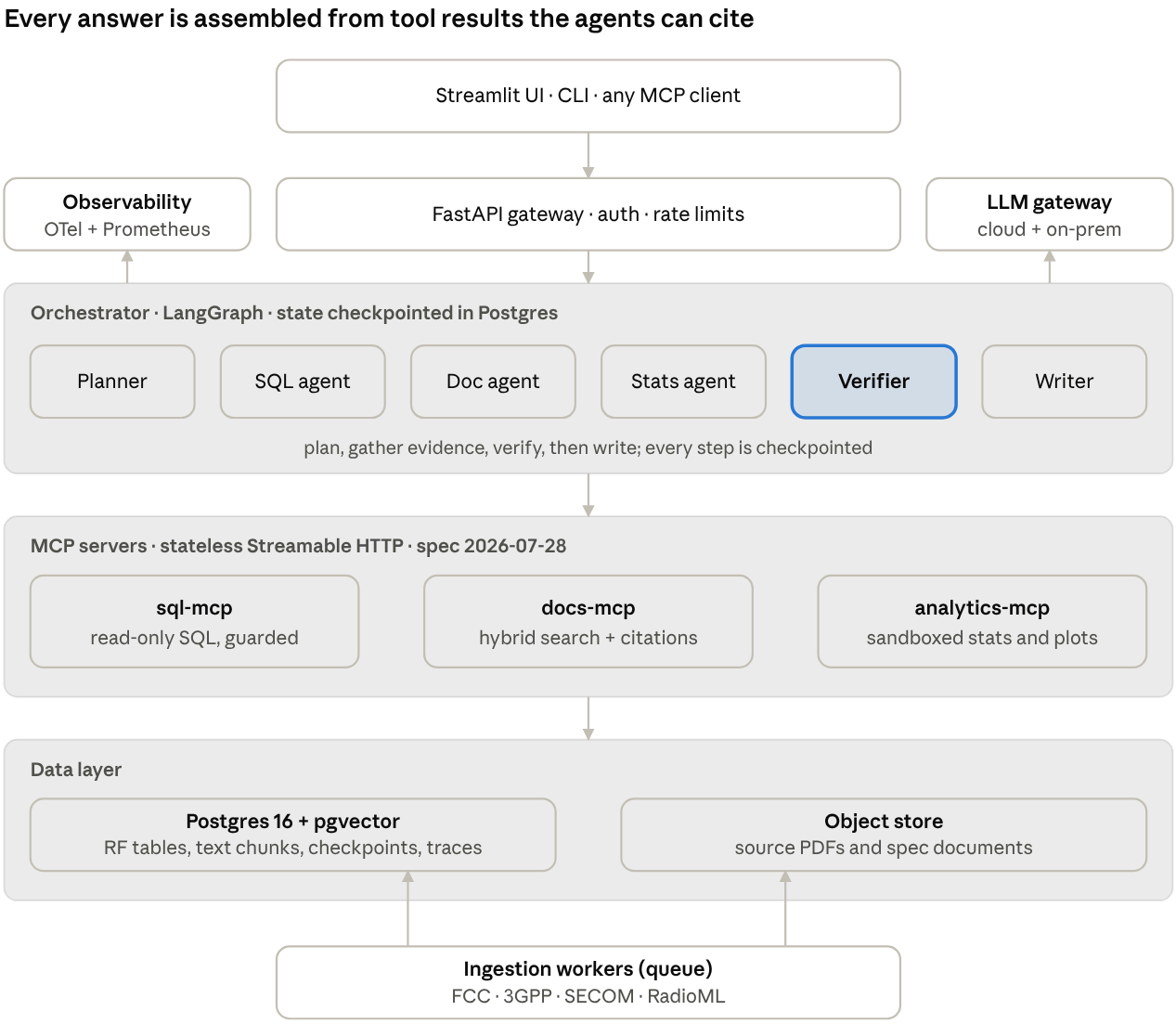
Source	What it holds	How to get it	Catch
FCC Equipment Authorization System	Grants per FCC ID: frequency range, output power, rule parts, equipment class. Exhibits include test reports, RF exposure (SAR) PDFs and photos	Public search at apps.fcc.gov/oetcf. The FCC also publishes open-data CSVs for grantee registrations and listed test firms	I found no official bulk download of the exhibit PDFs. Fetch a curated list of about 300 FCC IDs, rate-limited, after reading the FCC terms. Some exhibits are marked confidential
FCC Universal Licensing System	Wireless licence records: licensees, frequencies, services	Weekly full snapshots and daily transaction zips under data.fcc.gov/download/pub/uls/, pipe-delimited	Large. Load with COPY and start with one service category
3GPP specifications	LTE and 5G NR specs, including UE radio requirements	Free on the 3GPP server, in folders by release and series	Zip files of Word documents that you must convert. Spec numbers such as 36.101 and 38.101 are from my knowledge, so confirm the folders
RadioML 2018.01A (DeepSig)	About 2.5M I/Q samples, 24 modulation types, SNR from -20 to +30 dB	deepsig.ai/datasets	Check the licence before publishing anything derived from it; I believe it is non-commercial but did not verify. Use a subset

Source	What it holds	How to get it	Catch
UCI SECOM	1,567 production records, 590 sensor features, 104 fails (6.6%), timestamps	pip install ucimlrepo, then fetch_ucirepo(id=179)	A semiconductor fab, not RF. Sensors are anonymised and the set is small, so it cannot test scale

Scale-test data. For load tests only, replicate rows synthetically to tens of millions and label them synthetic. Keep that data out of every accuracy number.

Data card. Record each source's licence, access date and row counts, as the hackathon guides required.

4. Architecture



RFLens architecture · 5 layers, 6 agents, 3 MCP servers

Questions enter at the top, agents reach data only through the MCP servers, and every number or quote can be traced back to the database or the document store.

5. Agents and MCP servers

Six agents answer questions, and a separate offline job fills the tables they query. The team's own principle applies: add an agent only where a step needs one.

Agent	Job	Tools it calls
Planner	Splits the question into steps and picks the route; sees no data	None
SQL agent	Text-to-SQL using the schema resource and a few worked examples	sql-mcp
Doc agent	Retrieves passages and quotes them with chunk IDs	docs-mcp
Stats agent	Distributions, outliers, SECOM feature ranking, modulation accuracy against SNR	analytics-mcp
Verifier	Rejects any number not found in a SQL result and any quote not found in a chunk; allows two retries	Reads tool results only
Writer	Writes the final answer with citations	None

MCP servers

Server	Tools	Guardrails
sql-mcp	list_tables, describe_table, run_query	Read-only database role, allow-listed tables, row cap, statement timeout
docs-mcp	search_docs, get_chunk	Hybrid vector and keyword search, merged by reciprocal rank fusion
analytics-mcp	run_stats, plot	Sandboxed Python with no network and memory and time caps; returns artifact IDs

Report extraction (offline job). PDF to text and tables, then an LLM fills a Pydantic schema (band, test name, measured value, limit, unit, margin). Code checks units and recomputes margin as limit minus measured. A hand-labelled sample of about 50 reports measures accuracy before anything loads into Postgres. This job is the unique part of the project, so give it the most evaluation time.

Persistent state. The MCP revision dated 2026-07-28 removed protocol-level sessions, so servers handle each request on its own. That makes them easy to scale out. Keep investigation state in the LangGraph Postgres checkpointer, keyed by thread ID, so a run

can pause for approval and resume after a restart. Check which MCP revision your existing server targets: this one is a breaking change, with a required method header, a discovery call and OAuth 2.1 for remote servers.

6. RAG with citations and safe SQL

Both paths end the same way: every claim in the answer points to a SQL result or a document chunk that the Verifier has checked.

RAG pipeline

1. Convert sources to text: section-aware for 3GPP specs, table-aware for test reports.
2. Chunk with metadata: source, document ID, page, release, FCC ID.
3. Embed and store in pgvector next to the relational tables, with an HNSW index.
4. Retrieve with vector search and Postgres full-text search, merged by reciprocal rank fusion, then rerank.
5. Answer with chunk IDs. The Verifier opens each cited chunk and rejects claims it does not support.

One database for rows, vectors, checkpoints and traces keeps the system simple and matches the JD's Postgres and Aurora line. Measure it: if retrieval latency grows with corpus size, that is the point to compare against a dedicated vector store.

SQL guardrails

1. Connect with a read-only role that cannot run DDL or DML.
2. Parse every query with sqlglot. Allow only SELECT on an allow-list of tables and views.
3. Add a LIMIT, a statement timeout and a row cap on results.
4. Run EXPLAIN and reject queries above a cost threshold.
5. Log every query, its plan cost and the question that produced it.

Injection defence. Retrieved text is data. A passage that says "ignore your instructions" must not change tool calls, and the golden set includes planted examples to prove it.

7. Scale and production hardening

"Highly scalable" only counts if you measure it. The right column lists what to measure; I set no numeric targets, so fill in the real numbers and report them as they come out.

Concern	Design	What to measure
Throughput	Async FastAPI, a worker pool, parallel tool calls, a cache for repeated questions	p50 and p95 latency and requests per second under Locust or k6
Database	Partitioned measurement tables, indexes on band, frequency and FCC ID, connection pooling	Query p95 on 10M+ synthetic rows
Horizontal scale	Stateless API and MCP servers behind a load balancer, state in Postgres, Kubernetes manifests on kind or minikube	Throughput at 1 replica against N replicas
Model reliability	Timeouts, retries, fallback from one provider to the next, a token and cost budget per request	Fallback success rate with one provider switched off
On-prem path	An open model served by vLLM or Ollama behind the same interface	Golden-set accuracy gap against the cloud model
Security	API keys or OAuth, secrets in the environment, read-only database role, sandboxed code	Injection-test pass rate
Observability	OpenTelemetry traces across agent steps and MCP calls, Prometheus and Grafana, cost per request	A dashboard screenshot in the README
Delivery	CI runs lint, tests and the golden-set eval as a gate; one command starts the stack	A passing CI badge

The 2026-07-28 MCP revision defines how to propagate OpenTelemetry trace context between client and server, so one trace can follow a question from the UI through the agents into each tool.

8. Evaluation

Build a golden set of at least 100 questions and gate CI on it. Cover four types: SQL-only, document-only, mixed, and should-refuse (out of scope, unsafe SQL, planted injection).

Metric	How to measure it
SQL execution accuracy	The result set matches a hand-written reference query
Retrieval recall@5	The gold chunk appears in the top 5 results
Citation precision	Share of citations whose chunk supports the claim, checked by the Verifier and a manual sample
Extraction accuracy	Field-level F1 against about 50 hand-labelled test reports
Tool-call accuracy	The right tool is chosen with valid arguments
Refusal correctness	Out-of-scope and unsafe requests are refused
Injection resistance	Planted instructions in documents are ignored
Latency and cost	p50 and p95 latency, tokens and dollars per answer

Compare at least two configurations and choose with numbers: for example Claude against Gemini against a local model, with and without the reranker, or the base embedder against a fine-tuned one.

Fine-tuning, optional. The JD mentions it, so add it only after the pipeline works. The cheapest useful version fine-tunes a small embedding model on question and passage pairs generated from the specs and reports the recall@5 change. Keep it only if the number improves.

9. Build plan, risks and resume bullets

Six-week plan. Get one question answered end to end by the end of week 1, then widen.

1. Week 1: Postgres schema and migrations; loaders for ULS, SECOM and the FCC open-data CSVs; docker compose; CI skeleton.
2. Week 2: document pipeline for 3GPP specs and FCC reports into pgvector; hybrid search; docs-mcp.
3. Week 3: sql-mcp with guardrails; text-to-SQL; first 50 golden questions.
4. Week 4: LangGraph agents with the Postgres checkpointer, the Verifier and citations; Streamlit UI and the CLI.

5. Week 5: report-extraction job with the labelled sample; analytics-mcp over SECOM and RadioML.
6. Week 6: load tests, Kubernetes manifests, provider fallback, the on-prem run, the final eval report and README.

Risks and fallbacks

Risk	Fallback
FCC exhibit access or terms limit fetching	Use a smaller curated set, or the open-data CSVs plus any reports you can legitimately download by hand
Test-report PDFs vary in layout	Start with two test-lab formats and report accuracy per format
3GPP Word-to-text conversion is messy	Begin with two spec series and check chunks by eye
No GPU for the on-prem run	Use a small quantised model through Ollama and report the accuracy gap honestly
Scope creep	Cut fine-tuning and Kubernetes first; keep the golden set and the Verifier

Resume bullets to earn. Write these only once the numbers exist: "Built RFLens, an agentic RF data platform (LangGraph, FastAPI, Postgres with pgvector, three MCP servers); X% SQL execution accuracy and Y% citation precision on a Z-question golden set; p95 latency of N s at C concurrent users."

Sources

- FCC Equipment Authorization System and FCC open datasets
- FCC ULS downloads
- 3GPP specs download guidance
- UCI SECOM
- RadioML 2018.01A dataset description
- MCP 2026-07-28 changelog and MCP overview